TURTLEHERO

Entrega 2 – Algoritmos e Lógica de Programação

Projeto desenvolvido em Unity utilizando C#

Integrantes

Alessandro Galvão Silva
Murilo Ribeiro Vasconcellos
Valeska Affonso Marques

---

1 Introdução

Este documento apresenta a aplicação de estruturas e técnicas de lógica de programação no funcionamento do jogo TurtleHero, destacando os algoritmos utilizados e os resultados produzidos. Os conteúdos abordados e sua aplicação no projeto estão resumidos na Tabela 1.

Tabela 1 – Conteúdos e aplicação no projeto

Conteúdo	Aplicação no projeto
FOR e WHILE	Estruturas de repetição aplicadas à contagem de doces e à geração de notas.
Vetores	Armazenamento e associação entre teclas e pistas do jogo.
Modularização	Organização da lógica em funções e procedimentos.

2 Laço FOR

Arquivo: Assets/Scripts/Core/GameSession.cs

Aplicação: contagem dos doces liberados no jogo.

O método QuantidadeDoces() percorre todas as posições destinadas aos doces. O projeto possui 3 fases, com 4 doces por fase, totalizando 12 posições. Como essa quantidade é conhecida, o laço FOR é utilizado para percorrer os índices.

Trecho do algoritmo

```
public static int QuantidadeDoces()
{
    int quantidade = 0;
    for (int i = 0; i < TotalFases * DocesPorFase; i++)
    {
        if (DoceLiberado(i))
        {
            quantidade++;
        }
    }
    return quantidade;
}
```

Funcionamento: o contador *i* começa em zero e é incrementado a cada repetição. Enquanto for menor que o total de posições (12), o algoritmo verifica se o doce daquele índice está liberado. Se estiver, a variável *quantidade* é incrementada.

Resultado no projeto: o método retorna a quantidade de doces desbloqueados. O resultado pode variar de 0 a 12, conforme o progresso do jogador.

3 Laço WHILE

Arquivo: Assets/Scripts/Notas/NoteSpawner.cs

Aplicação: geração contínua de notas no modo aleatório.

O método SpawnLoop() mantém a criação de notas enquanto a variável running for verdadeira. A repetição depende de uma condição, característica do laço WHILE.

Trecho do algoritmo

```
IEnumerator SpawnLoop()
{
    if (startDelay > 0f)
    {
        yield return new WaitForSeconds(startDelay);
    }
    while (running)
    {
        yield return new WaitForSeconds(
            Random.Range(minInterval, maxInterval)
        );
        if (!running)
        {
            yield break;
        }
        int lane = GetNextLane();
        if (lane < 0)
        {
            yield break;
        }
        Spawn(lane);
    }
}
```

Funcionamento: após o atraso inicial, o WHILE verifica a condição *running*. Enquanto ela for verdadeira, o algoritmo aguarda um intervalo, identifica a próxima pista e solicita a criação de uma nota. Se a execução for interrompida ou não houver pista disponível, a rotina é encerrada.

Resultado no projeto: as notas são geradas em intervalos configurados durante a partida. Quando o sistema interrompe a geração, a condição deixa de ser verdadeira e o laço termina.

4 Vetores

Arquivo: Assets/Scripts/Combate/HitZone.cs

Aplicação: armazenamento das teclas de controle e associação com as pistas.

O sistema de acerto utiliza dois vetores: um armazena as teclas que o jogador pode pressionar e o outro armazena o número da pista correspondente a cada tecla. Os elementos são relacionados pela posição (índice) que ocupam nos vetores.

Vetores declarados no projeto

```
public Key[] teclas = { Key.D, Key.F, Key.K, Key.L };
public int[] pistaDaTecla = { 3, 2, 1, 0 };
```

Tabela 2 – Associação entre teclas e pistas por índice

Índice	Elemento em teclas[]	Elemento em pistaDaTecla[]
0	`Key.D`	`3`
1	`Key.F`	`2`
2	`Key.K`	`1`
3	`Key.L`	`0`

Funcionamento: o método Update() percorre o vetor *teclas* por meio de um FOR. Quando identifica uma tecla pressionada, utiliza o índice correspondente para consultar a pista em *pistaDaTecla*. Dessa forma, os dois vetores mantêm a associação entre tecla e pista.

Exemplo de leitura: quando o jogador pressiona a tecla F, o índice 1 é utilizado para consultar o valor 2 no vetor de pistas. Assim, o sistema identifica a pista associada à tecla pressionada.

Resultado no projeto: as teclas D, F, K e L são reconhecidas e associadas às quatro pistas do jogo, permitindo identificar em qual pista o jogador tentou acertar uma nota.

5 Modularização

Arquivo: Assets/Scripts/Core/GameSession.cs

Aplicação: separação das tarefas de pontuação, controle da fase e liberação de doces em métodos específicos.

A classe GameSession organiza a lógica da partida em métodos. Cada método concentra uma responsabilidade, permitindo que outras partes do jogo solicitem uma operação sem precisar repetir toda a lógica. A Tabela 3 apresenta exemplos de métodos do projeto.

Tabela 3 – Métodos da classe GameSession

Método	Tipo	Responsabilidade
`Registrar()`	Função (int)	Calcula e retorna os pontos ganhos a partir do julgamento da nota.
`QuantidadeDoces()`	Função (int)	Conta e retorna a quantidade de doces liberados.
`ConcluirFase()`	Procedimento (void)	Atualiza a pontuação total e libera os doces da fase concluída.
`LiberarDocesDaFase()`	Procedimento (void)	Marca os doces da fase como liberados.

Exemplo de procedimento

```
public static void ConcluirFase(int fase)
{
    PontosTotal += Pontos;
    LiberarDocesDaFase(fase);
    FaseRecemVencida = fase;
}
```

Funcionamento: ao concluir uma fase, o método atualiza a pontuação total, chama o método responsável por liberar os doces e registra qual fase foi concluída. Como não retorna um valor, ConcluirFase() é um procedimento. Já QuantidadeDoces() e Registrar() retornam valores inteiros, sendo funções.

Resultado no projeto: as responsabilidades ficam separadas em métodos menores. Isso facilita a leitura, a manutenção e a reutilização da lógica durante a execução do jogo.

6 Resultados no projeto

Os algoritmos apresentados estão relacionados a funcionalidades do TurtleHero e contribuem para o funcionamento da partida. A Tabela 4 resume os resultados observados na lógica de cada conteúdo.

Tabela 4 – Resumo dos resultados por conteúdo

Conteúdo	Aplicação	Resultado observado na lógica
FOR	Percorre as posições dos doces.	Obtém a quantidade de doces desbloqueados, entre 0 e 12.
WHILE	Controla a geração aleatória de notas.	Mantém a geração ativa enquanto a condição running for verdadeira.
Vetores	Armazena teclas e pistas associadas.	Relaciona cada tecla de controle à pista correspondente.

Conteúdo	Aplicação	Resultado observado na lógica
Modularização	Divide tarefas em funções e procedimentos.	Organiza pontuação, conclusão de fase e liberação de doces em métodos próprios.

7 Considerações finais

A implementação dos conceitos de FOR, WHILE, vetores e modularização no TurtleHero demonstra a aplicação de estruturas fundamentais da lógica de programação em situações do próprio jogo. O FOR percorre uma quantidade definida de posições; o WHILE mantém uma repetição condicionada; os vetores organizam dados relacionados; e a modularização distribui as tarefas entre funções e procedimentos.

Os trechos selecionados destacam a lógica utilizada nas funcionalidades apresentadas, com estruturas explícitas e sem operador ternário nos exemplos de código deste documento.

Arquivos de referência do projeto

- `Assets/Scripts/Core/GameSession.cs`
- `Assets/Scripts/Notas/NoteSpawner.cs`
- `Assets/Scripts/Combate/HitZone.cs`